

Билет 20. Memory reclamation в lock-free структурах. `std::atomic_shared_ptr`

0. Формулировка билета

Проблема управления памятью в lock-free структурах. Обзор методов решения проблемы. Класс `std::atomic_shared_ptr`. В чём отличие от обычного `shared_ptr`, в чём идея реализации? Можно ли сделать lock-free `atomic_shared_ptr`?

Главная идея:

В lock-free структурах трудно не только правильно изменить указатели через CAS, но и понять, когда можно безопасно удалить узел.

В обычной структуре с mutex всё проще:

```
std::mutex m;

void pop() {
    std::lock_guard lock(m);

    Node* old = head;
    head = head->next;

    delete old;
}
```

Почему это безопасно?

Потому что пока поток держит mutex, другие потоки не могут одновременно работать со структурой.

В lock-free структуре mutex нет.

Другой поток мог раньше прочитать указатель на узел и ещё продолжать с ним работать.

Поэтому простой `delete` может дать:

```
use-after-free
ABA
повреждение структуры
редкие трудноуловимые падения
```

1. Где возникает проблема

Возьмём стек Трайбера.

Упрощённый `pop`:

```
Node* old_head = head.load();

while (old_head != nullptr) {
    Node* next = old_head->next;

    if (head.compare_exchange_weak(old_head, next)) {
        T value = old_head->value;
        delete old_head;
        return value;
    }
}
```

На первый взгляд всё нормально:

```
мы CAS-ом сняли old_head со стека
значит можно delete old_head
```

Но это неверно.

2. Use-after-free сценарий

Сценарий:

```
thread A:
    old_head = head.load()
    // old_head указывает на узел X
    pause

thread B:
    pop()
    CAS успешно снимает X
    delete X

thread A:
    продолжает выполнение
    читает old_head->next
```

Но `old_head` уже указывает на освобождённую память.

Итог:

```
use-after-free  
undefined behavior
```

Важная мысль:

Узел может быть логически удалён из структуры, но физически его ещё нельзя освободить, пока какой-то поток мог ранее прочитать указатель на него.

3. Почему mutex решает это автоматически

С mutex:

```
lock(m);  
old = head;  
head = head->next;  
delete old;  
unlock(m);
```

Другой поток не может одновременно:

```
прочитать head  
зависнуть  
потом использовать старый указатель
```

потому что все операции идут под одним lock.

Mutex даёт простую гарантию:

```
если поток работает со структурой,  
другой поток не может параллельно менять и удалять её узлы
```

В lock-free структуре такой глобальной защиты нет.

4. Две разные задачи

В lock-free структурах надо решать две задачи.

1. Логическое удаление

Например:

```
CAS(head, old_head, old_head->next)
```

После успешного CAS узел больше не является частью структуры.

Он логически удалён.

2. Физическое освобождение памяти

```
delete old_head;
```

Это можно делать только когда точно известно:

ни один поток больше не держит указатель на этот узел
и не будет его читать

Memory reclamation — это как раз про вторую задачу.

5. Почему нельзя просто не удалять

Можно сказать:

не будем delete-ить узлы вообще

Это действительно убирает use-after-free.

Но тогда будет:

memory leak
неограниченный рост памяти
невозможность долгой работы программы

Для теста или короткой программы иногда так делают, но для настоящей структуры данных нужен способ безопасного освобождения.

6. Требования к memory reclamation

Хорошая схема должна:

не удалять узел, пока кто-то может его использовать
в итоге освобождать удалённые узлы

не ломать lock-free прогресс структуры
иметь разумный overhead

Идеального решения нет.

Разные методы дают разные компромиссы:

hazard pointers
epoch-based reclamation
reference counting
garbage collection
atomic_shared_ptr

7. Hazard pointers

Hazard pointer — это способ, при котором поток явно публикует указатель на узел, с которым он сейчас работает.

Идея:

thread A говорит всем:
я сейчас работаю с узлом X
не удаляйте X

Для каждого потока есть один или несколько hazard pointer slots.

Например:

```
std::atomic<Node*> hazard[MAX_THREADS];
```

Поток перед использованием узла делает:

1. Читает head.
2. Записывает head в свой hazard pointer.
3. Перечитывает head и проверяет, что он не изменился.
4. Только после этого безопасно читает поля узла.

Почему нужно перечитать?

Потому что между чтением `head` и публикацией hazard pointer другой поток мог уже удалить этот узел.

8. Схема hazard pointer при pop

Упрощённо:

```
loop:
    old_head = head.load()

    publish old_head in my hazard pointer

    if head.load() != old_head:
        retry

    // теперь old_head защищён
    next = old_head->next

    if CAS(head, old_head, next) successful:
        clear hazard pointer
        retire old_head
        return value
```

Важно:

```
retire old_head
```

не значит сразу `delete`.

Узел кладётся в список “удалённых, но пока не освобождённых”.

Периодически поток сканирует hazard pointers всех потоков.

Если retired-узел не находится ни в одном hazard pointer, его можно удалить.

9. Плюсы и минусы hazard pointers

Плюсы:

```
точная защита конкретных узлов
можно освобождать память без глобальных эпох
подходит для структур, где поток держит небольшое число указателей
```

Минусы:

```
нужно публиковать hazard pointer перед доступом к узлу
нужно сканировать hazard pointers всех потоков
```

overhead на атомарные записи/чтения
сложно правильно реализовать

На экзамене достаточно сказать:

Hazard pointer — это публичное объявление “я сейчас использую этот узел”. Пока узел в hazard pointer хотя бы одного потока, его нельзя физически удалять.

10. Epoch-based reclamation

Epoch-based reclamation — схема с эпохами.

Идея:

Мы не отслеживаем конкретный узел у конкретного потока.
Мы отслеживаем, в какой эпохе находятся потоки.

Есть глобальная эпоха:

global_epoch

Поток, входя в lock-free операцию, объявляет:

я активен в текущей эпохе

Удалённые узлы не освобождаются сразу.

Они помещаются в retired list с номером эпохи.

Освобождать узлы можно, когда все потоки вышли из старых эпох.

11. Почему epochs работают

Сценарий:

epoch = 10

thread A:

вошёл в операцию в epoch 10
мог прочитать указатель на узел X

thread B:

удалил X
положил X в retired list epoch 10

Нельзя удалять X, пока thread A всё ещё активен в epoch 10.

Когда все потоки, которые были активны в epoch 10, вышли из операций, можно считать:

никто уже не держит старые указатели из epoch 10

Тогда retired-узлы из старых эпох можно освобождать.

12. Плюсы и минусы epoch-based reclamation

Плюсы:

часто быстрее hazard pointers
нет необходимости защищать каждый узел отдельно
хорошо работает, если операции короткие

Минусы:

если поток завис внутри эпохи, освобождение памяти может остановиться
память может долго не возвращаться
нужно аккуратно отмечать вход/выход из критической области

Главная проблема:

Один остановленный поток может задерживать освобождение большого количества памяти.

Это не обязательно ломает lock-free прогресс операций, но может привести к росту памяти.

13. Reference counting

Можно хранить счётчик ссылок на узел.

Идея:

поток хочет работать с узлом
увеличивает reference count

закончил
уменьшает reference count

если узел логически удалён и `refcount == 0`:
можно `delete`

Это похоже на `shared_ptr`.

Плюсы:

понятная идея
узел удаляется, когда на него нет ссылок

Минусы:

каждый доступ требует атомарного изменения счётчика
дорого
может быть сложно избежать гонок между чтением указателя и увеличением счётчика
может быть не lock-free

14. Garbage collection

В языках с garbage collector проблема проще.

Если объект ещё достижим из какого-то потока, GC его не удалит.

Это помогает против:

use-after-free
переиспользования адресов
части ABA-сценариев

Но GC есть не во всех языках.

В C++ автоматического tracing GC нет, поэтому memory reclamation приходится решать явно.

15. ABA и memory reclamation

Memory reclamation тесно связана с ABA.

ABA:

```
head был A
потом стал B
потом снова стал A
CAS видит A и думает, что ничего не изменилось
```

Особенно опасный вариант:

```
узел A удалили
память освободили
аллокатор выдал тот же адрес новому узлу
указатель снова имеет значение A
```

CAS сравнивает только адрес.

Он не понимает, что по этому адресу уже другой объект.

Memory reclamation помогает, потому что если мы не освобождаем и не переиспользуем узел слишком рано, то уменьшается риск такого ABA.

Но важно:

Memory reclamation не всегда полностью решает логическую ABA, если тот же самый узел реально удалили и потом вставили обратно.
Для полного решения часто используют tagged pointers или версии.

16. Tagged pointers

Другой способ борьбы с ABA — хранить вместе с указателем счётчик версии.

Вместо:

```
std::atomic<Node*> head;
```

используем логически:

```
atomic<pair<Node*, counter>>
```

Например:

```
(A, 1)
(B, 2)
(A, 3)
```

Даже если указатель снова стал `A`, версия изменилась.

CAS сравнивает пару:

```
(A, 1) != (A, 3)
```

Проблема:

нужен атомарный CAS по паре `pointer + counter`
иногда нужен `double-width CAS`

17. `std::shared_ptr`

Обычный `std::shared_ptr<T>` — умный указатель с подсчётом ссылок.

У него есть:

указатель на объект
указатель на `control block`

В `control block` хранится:

`strong reference count`
`weak reference count`
`deleter`
`allocator`, иногда другая служебная информация

Когда копируем `shared_ptr`:

`strong count` увеличивается

Когда `shared_ptr` уничтожается:

`strong count` уменьшается

Когда `strong count` стал 0:

объект удаляется

Когда `strong` и `weak count` стали 0:

удаляется control block

18. Обычный `shared_ptr` и многопоточность

Важное различие.

Разные объекты `shared_ptr`, которые владеют одним и тем же объектом, можно копировать/уничтожать из разных потоков: control block внутри потокобезопасно обновляет счётчики.

Например:

```
std::shared_ptr<T> p1 = std::make_shared<T>();
std::shared_ptr<T> p2 = p1;

// p1 и p2 – разные shared_ptr objects.
// Их можно использовать в разных потоках.
```

Но один и тот же объект `shared_ptr` нельзя одновременно читать и писать из разных потоков без синхронизации.

Плохо:

```
std::shared_ptr<T> global_p;

thread 1:
    global_p = std::make_shared<T>();

thread 2:
    auto local = global_p;
```

Это data race на самом объекте `global_p`, если нет синхронизации.

То есть:

```
control block потокобезопасен
сам объект shared_ptr как переменная – нет
```

19. Зачем нужен `atomic shared_ptr`

Нам хочется атомарно делать операции с самой переменной типа shared pointer:

```
std::atomic<std::shared_ptr<T>> p;
```

или в старой терминологии:

```
atomic_shared_ptr<T>
```

Нужно уметь:

```
atomic load shared_ptr  
atomic store shared_ptr  
compare_exchange shared_ptr
```

Например, для lock-free структуры:

```
std::atomic<std::shared_ptr<Node>> head;
```

Тогда поток может сделать:

```
auto p = head.load();
```

И пока локальный `shared_ptr p` жив, объект `Node` не будет удалён.

Это решает часть проблемы memory reclamation:

Если поток атомарно загрузил `shared_ptr` на узел, счётчик ссылок увеличен, и узел нельзя удалить, пока локальная копия существует.

20. Отличие `atomic_shared_ptr` от обычного `shared_ptr`

Обычный `shared_ptr`:

безопасно управляет lifetime объекта через control block
но сама переменная `shared_ptr` не является атомарной

`atomic_shared_ptr` / `std::atomic<std::shared_ptr<T>>`:

позволяет атомарно load/store/CAS сам `shared_ptr`
и одновременно корректно управляет reference count

То есть отличие:

```
shared_ptr:  
    атомарны счётчики в control block  
  
atomic_shared_ptr:  
    атомарен доступ к самой shared_ptr-переменной
```

Пример:

```
std::atomic<std::shared_ptr<Node>> head;  
  
auto old = head.load();  
  
auto new_node = std::make_shared<Node>();  
new_node->next = old;  
  
head.compare_exchange_weak(old, new_node);
```

21. Почему `atomic_shared_ptr` сложнее обычного `atomic pointer`

С обычным атомарным указателем:

```
std::atomic<Node*> p;
```

load — это просто загрузка адреса.

С `atomic shared pointer` load должен сделать больше:

```
атомарно прочитать указатель/control block  
увеличить reference count  
вернуть shared_ptr
```

Проблема:

Нельзя просто сначала прочитать сырой указатель, а потом увеличить refcount, потому что объект могли удалить между этими действиями.

Нужно, чтобы операция атомарной загрузки `shared_ptr` была согласована с изменением счётчика ссылок.

22. Идея реализации atomic_shared_ptr

Одна из идей реализации:

внутри есть atomic pointer на control block / counted pointer
операции load/store/CAS защищают момент получения ссылки

При `load` нужно гарантировать:

если мы получили shared_ptr на объект,
то strong count был увеличен до того,
как объект мог быть уничтожен

При `store`:

новый shared_ptr публикуется атомарно
старый shared_ptr корректно отпускается

При `compare_exchange`:

сравнивается текущий shared_ptr с expected
если совпал – меняется на desired
если не совпал – expected обновляется текущим значением
и reference counts корректно меняются

На практике реализация может использовать внутренний lock или специальные атомарные алгоритмы.

23. Можно ли сделать lock-free atomic_shared_ptr?

Короткий ответ:

Теоретически можно построить неблокирующую схему, но стандартный
`std::atomic<std::shared_ptr<T>>` не обязан быть lock-free.

В C++ у атомиков есть метод:

`p.is_lock_free()`

Для `std::atomic<std::shared_ptr<T>>` результат зависит от реализации.

Почему трудно сделать lock-free?

Потому что надо атомарно управлять несколькими вещами:

```
указателем на объект  
указателем на control block  
strong reference count  
операциями CAS/load/store
```

Обычный CAS по одному машинному слову часто недостаточен.

Возможные варианты реализации:

```
использовать внутренний mutex/spinlock  
использовать tagged/counted pointer  
использовать double-width CAS, если платформа поддерживает  
использовать специальный control block
```

Поэтому на экзамене хорошая фраза:

`atomic_shared_ptr` удобен для lifetime, но не гарантирует lock-free реализацию;
надо проверять `is_lock_free()` и понимать overhead reference counting.

24. Atomic shared_ptr как memory reclamation

Почему `atomic_shared_ptr` помогает?

Потому что если поток сделал:

```
auto p = head.load();
```

то теперь у него есть owning-ссылка на узел.

Пока `p` жив:

```
узел не будет уничтожен
```

Это убирает use-after-free для этого узла.

Можно писать структуры, где узлы связаны через `shared_ptr` :

```
struct Node {  
    T value;
```



```
std::shared_ptr<Node> next;  
};
```

Но это часто дорого:

```
много атомарных inc/dec reference count  
сложнее CAS  
больше аллокаций  
хуже cache locality  
не всегда lock-free
```

Поэтому `atomic_shared_ptr` — не бесплатная замена hazard pointers/epochs.

25. Сравнение методов

Hazard pointers

поток публикует конкретный указатель, который он использует
удаление откладывается, пока указатель есть в hazard set

Хорошо:

точно защищает отдельные узлы
не требует shared_ptr на каждый узел

Плохо:

нужно сканировать hazard pointers
сложно реализовать

Epoch-based reclamation

узлы удаляются не сразу, а после завершения старых эпох

Хорошо:

часто быстро
мало overhead на каждую операцию

Плохо:

зависший поток может задерживать освобождение памяти

Reference counting / shared_ptr

узел жив, пока есть ссылки

Хорошо:

понятная модель lifetime

Плохо:

дорогие атомарные счётчики
сложно сделать lock-free

Tagged pointers

помогают против ABA через version counter

Хорошо:

CAS видит не только указатель, но и версию

Плохо:

нужна поддержка CAS по расширенному значению
не решает всю memory reclamation сам по себе

26. Что сказать коротко

В lock-free структурах есть отдельная проблема управления памятью. После успешного CAS узел может быть логически удалён из структуры, но его нельзя сразу физически освободить через `delete`: другой поток мог раньше прочитать указатель на этот узел и ещё использовать его. Поэтому наивный `delete` приводит к use-after-free.

Основные методы memory reclamation: hazard pointers, epoch-based reclamation, reference counting, garbage collection, atomic shared pointers. Hazard pointer означает, что поток публично объявляет "я сейчас использую этот узел", и пока указатель опубликован, узел нельзя удалять.

Epoch-based reclamation откладывает удаление узлов до момента, когда все потоки покинули старые эпохи. Reference counting удаляет объект, когда счётчик ссылок стал нулём, но требует дополнительных атомарных операций.

Обычный `std::shared_ptr` потокобезопасно управляет control block и счётчиками ссылок, но сама переменная `shared_ptr` не является атомарной: если один поток пишет в тот же `shared_ptr`, а другой читает, нужна синхронизация. `std::atomic<std::shared_ptr<T>>` или `atomic_shared_ptr` позволяет атомарно делать `load`, `store`, `compare_exchange` над самой shared pointer-переменной и корректно обновлять reference count.

Идея `atomic_shared_ptr` как reclamation — если поток атомарно загрузил `shared_ptr` на узел, то счётчик ссылок увеличен, и узел не будет уничтожен, пока локальная копия `shared_ptr` жива. Но это не бесплатно: операции дороже обычных atomic pointer, из-за reference counting, control block и возможных внутренних lock-ов. Стандартный `std::atomic<std::shared_ptr<T>>` не обязан быть lock-free; это можно проверить через `is_lock_free()`. Теоретически lock-free реализация возможна на некоторых платформах, но в общем случае она сложна.

27. Мини-проверка

1. Почему в lock-free структурах нельзя просто сразу делать `delete` удалённого узла?
2. Чем логическое удаление отличается от физического освобождения памяти?
3. Какой use-after-free сценарий возникает в Treiber stack или Michael-Scott queue?
4. Что такое hazard pointer?
5. Почему после публикации hazard pointer нужно перепроверить, что указатель всё ещё актуален?
6. Что такое retired list?
7. Как работает epoch-based reclamation?
8. Главный минус epoch-based reclamation?
9. Чем memory reclamation связана с ABA?
10. Чем обычный `shared_ptr` отличается от `atomic_shared_ptr`?
11. Почему `atomic_shared_ptr` помогает с lifetime узлов?
12. Обязан ли `std::atomic<std::shared_ptr<T>>` быть lock-free?